


①

def print_square(n):

for i in range(1, n+1):

line = '*' * n $\Theta(n)$ print(line) $\Theta(1)$

>>> print_square(4)

```
****
****
****
****
```

$$T(n) = \Theta(n^2) = (\Theta(n) + 1)_n$$

②

def print_triangle(n):

for i in range(1, n+1):

line = '*' * i $\Theta(i)$ print(line) $\Theta(1)$

>>> print_triangle(4)

```
*
**
***
****
```

$$T(n) = 1 + 2 + 3 + 4 + 5 + \dots + n$$

$$= \frac{n(n+1)}{2} = \cancel{\Theta(n^2)} + \cancel{\Theta(n)} = \Theta(n^2)$$

③

def prefix_average(lst):

n = len(lst) $\# \Theta(1)$ result = [0] * n $\# \Theta(n)$

for i in range(n):

curr_sum = sum(lst[0:i+1]) $\# \Theta(i)$ curr_avg = curr_sum / (i+1) $\# \Theta(1)$ result[i] = curr_avg $\# \Theta(1)$ return result $\# \Theta(1)$

>>> print(prefix_average([10, 20, 30, 40, 50]))

[10.0, 15.0, 20.0, 25.0, 30.0]

$$T(n) = \cancel{\Theta(1)} + \cancel{\Theta(n)} + \Theta(n^2) = \Theta(n^2)$$

④

def prefix_average(lst):

n = len(lst) $\# \Theta(1)$ result = [0] * n $\# \Theta(n)$ curr_sum = 0 $\# \Theta(1)$

for i in range(n):

curr_sum += lst[i] $\# \Theta(1)$ curr_avg = curr_sum / (i+1) $\# \Theta(1)$ result[i] = curr_avg $\# \Theta(1)$ return result $\# \Theta(1)$

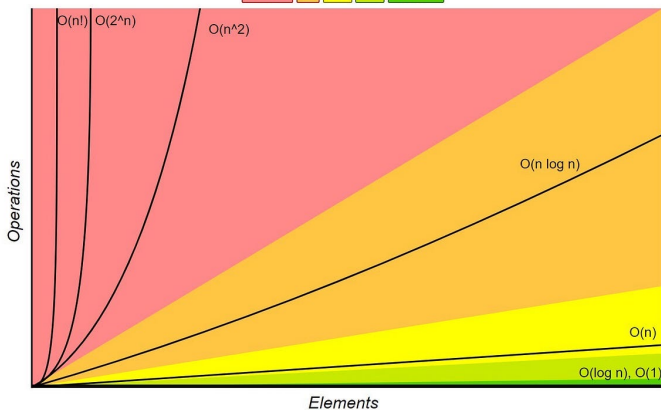
>>> print(prefix_avg([10, 20, 30, 40, 50]))

$$T(n) = \Theta(n) + \Theta(n) + \Theta(1) = \Theta(n)$$

↑ creating list of 0s
↑ for-loop
↑ len variable return

Big-O Complexity Chart

Horrible Bad Fair Good Excellent



Searching Algorithms

Linear search

1	4	3	8	7	2	5	9	0
---	---	---	---	---	---	---	---	---

def linear_search(lst, val):

$$T(n) = \Theta(n) + \Theta(1) = \Theta(n)$$

$\Theta(1) = \Theta(n)$

```

    for i in range(len(lst)):
        if lst[i] == val: #  $\Theta(1)$ 
            return i #  $\Theta(1)$ 
    return None #  $\Theta(1)$ 

```

Sorted search

0	1	2	3	4	5	7	8	9
---	---	---	---	---	---	---	---	---

$$\Theta\left(\frac{n}{2}\right)$$

Sorted search

0	1	2	3	4	5	7	8	9
---	---	---	---	---	---	---	---	---

$$\Theta\left(\frac{n}{4}\right)$$

$$1 \rightarrow \frac{n}{1} = \frac{n}{2^0}$$

$$2 \rightarrow \frac{n}{2} = \frac{n}{2^1}$$

$$3 \rightarrow \frac{n}{4} = \frac{n}{2^2}$$

$$4 \rightarrow \frac{n}{8} = \frac{n}{2^3}$$

...

$$k \rightarrow \frac{n}{2^{k-1}}$$

$$\frac{n}{2^{k-1}} = 1$$

$$n = 2^{k-1}$$

$$\log_2(n) = \log_2(2^{k-1})$$

$$\log_2(n) = (k-1) \log_2(2)$$

$$\log_2(n) = (k-1)$$

$$k = \log_2(n) + 1$$

$$\Theta\left(\frac{n}{8}\right)$$

$$\Theta\left(\frac{n}{16}\right)$$

$$\Theta\left(\frac{n}{32}\right)$$

$$\log_c(x) = \log_c(y) / \log_c(y)$$

$$T(k) = \log_2(n) + 1 = \Theta(\log_2(n))$$

```
def binary_search(list, val):
```

```
    start = 0
```

```
    stop = len(list) - 1
```

```
    while start <= stop:
```

```
        mid = (start + stop) // 2
```

```
        if list[mid] == val:
```

```
            return mid
```

```
        elif list[mid] > val:
```

```
            stop = mid - 1
```

```
        else:
```

```
            start = mid + 1
```

```
    return None
```

1	2	3	4	5	6	7	8	9
---	---	---	---	---	---	---	---	---